

Introducción

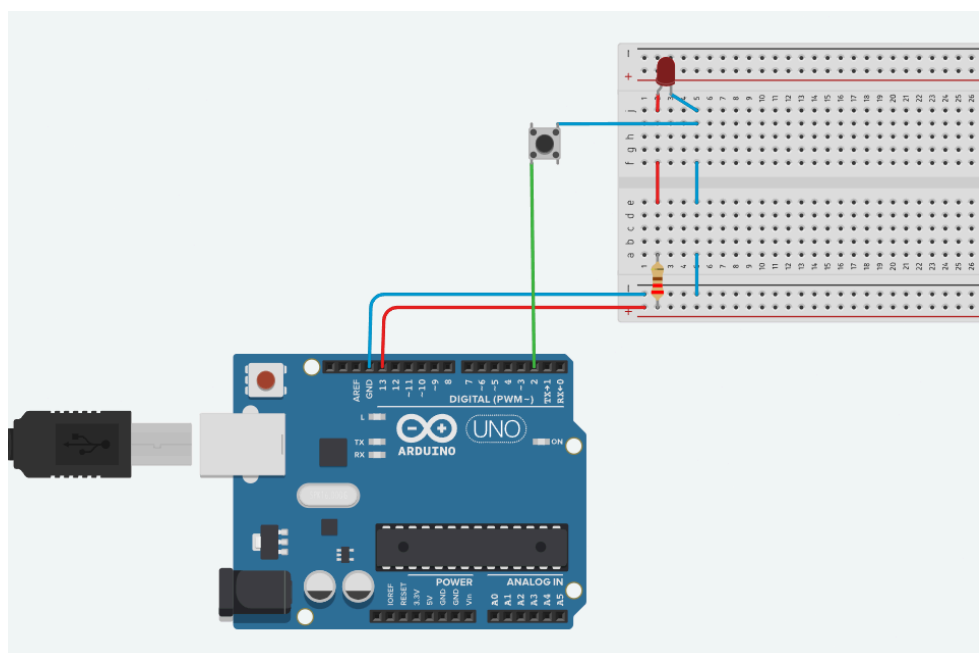
El objetivo de este sistema fue crear un circuito controlado por un Arduino conformado por un foco LED y un pulsador en donde el foco LED se mantuviera encendido mientras que el pulsador no esté siendo presionado. En caso de ser presionado, el LED se debería mantener apagado durante la duración de la pulsación, solo prentiendo una vez el pulsador se libere.

Metodología

Materiales

- Arduino Uno R3 SMD
- Protoboard
- 1 LED rojo
- 1 resistor de 220 Ω
- 1 pulsador
- Cables jumper

El Arduino fue utilizado para poder ejecutar el programa escrito dentro del circuito elaborado. Por otro lado, el protoboard era necesario para poder posicionar y conectar el resto de los componentes de manera sencilla. Dado el objetivo de este proyecto, el LED era necesario para mostrar el funcionamiento del código, mostrando que el pulsador podía mantenerlo prendido y apagado. Esto también se puede decir del pulsador en sí, siendo este requerido para la activación del programa. Por último, la resistencia fue utilizada para prevenir que el LED u otros componentes se quemarán y los cables jumper para conectar todos los componentes previamente mencionados.



Los componentes fueron conectados de esta manera en sentido que facilitarán las conexiones físicas. Primeramente, se escogieron los pines digitales 13 y 2 debido a la necesidad de programar solamente dos estados (prendido y apagado) y la conveniencia de su posicionamiento en los extremos del Arduino. La resistencia fue colocada donde se observa para omitir el uso de un cable adicional. El botón, por otro lado, se ve conectado de manera diferente al circuito físico dentro del diagrama anterior. Esto se debe a limitaciones del programa Tinkercad, en el cual no cabe el botón dentro del protoboard con suficiente espacio para sacar las dos conexiones extra necesarias para completar el circuito. En el circuito construido el botón se encuentra posicionado adecuadamente sobre el protoboard. Además, la resistencia utilizada fue de 10 kΩ, ya que a pesar de que esta esté muy por sobre el valor necesario, las demás disponibles tenían el problema inverso, siendo demasiado débiles y, por ende, poniendo el riesgo al LED de quemarse.

Código utilizado

```
1  const int pinLED = 13;
2  const int pinBOTON = 2;
3  void setup(){
4      pinMode(pinBOTON,INPUT_PULLUP);
5      pinMode(pinLED,OUTPUT);
6  }
7  void loop(){
8      int estadoBoton = digitalRead(pinBOTON);
9      if(estadoBoton == LOW){
10         digitalWrite(pinLED,LOW);
11     }else{
12         digitalWrite(pinLED,HIGH);
13     }
14 }
```

Explicacion del codigo

```
1  const int pinLED = 13;
2  const int pinBOTON = 2;
```

Como dicho anteriormente, se utilizaron los pines 13 y 2. Se utilizaron por medio de constantes para poder nombrar la funcionalidad de los pines y tener una mejor comprensión del código.

```
3  void setup(){
4      pinMode(pinBOTON,INPUT_PULLUP);
5      pinMode(pinLED,OUTPUT);
```

Dentro del setup del sketch, se utilizó el operador “void” para indicarle al programa que no requería de un registro de datos. Además, dentro del setup se definieron las funcionalidades de las constantes previas; el pin 2 se definió como una entrada usando la constante

preconfigurada "INPUT PULLUP". El pin 13 fue definido como un valor de salida con "OUTPUT", permitiendo que el LED se alumbré.

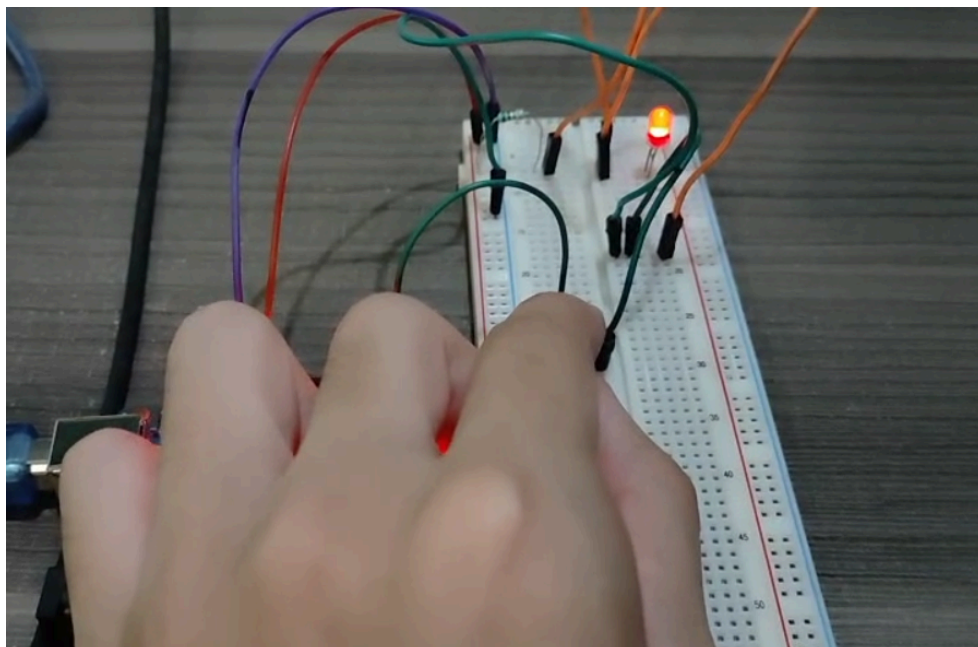
```
7  void loop(){
8      int estadoBoton = digitalRead(pinBOTON);
9      if(estadoBoton == LOW){
10         digitalWrite(pinLED,LOW);
11     }else{
12         digitalWrite(pinLED,HIGH);
13     }
```

Dentro del loop del sketch, se creó una variable que refleja el valor del pin 2; estos valores pueden ser HIGH o LOW. Luego, se abrió una estructura de control condicional donde si el valor leído en la variable "estadoBoton" fuera LOW, entonces el LED dejaría de brillar. En el caso de que no se cumpla la condición de la lectura de "estadoBoton", el LED seguiría brillando de manera ininterrumpida. Todo este código funciona ya que el pulsador es normalmente abierto.

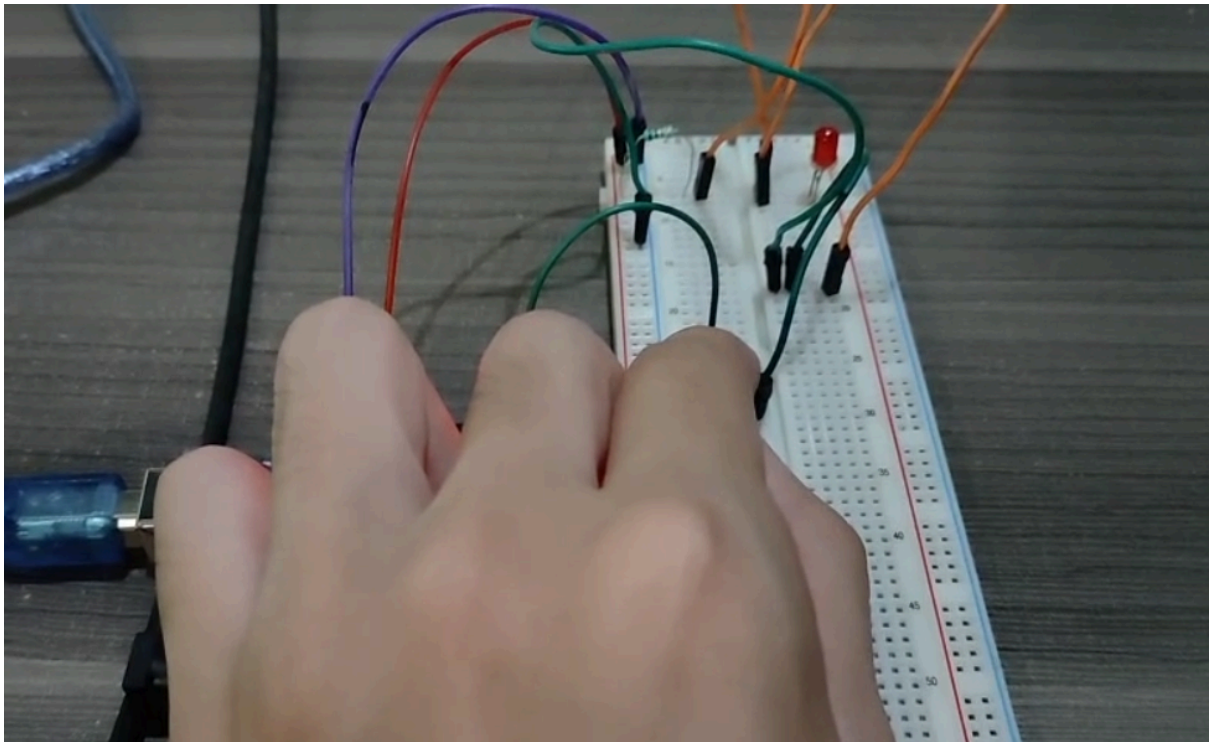
Resultados

Una vez el sistema conectado y el programa subido al Arduino, se probó su funcionamiento. Los resultados de este dieron lo siguiente:

- Cuando el pulsador no estaba siendo presionado, el LED permanecía prendido.



- Cuando el pulsador estaba siendo presionado, el LED permanecía apagado.



- Cuando el pulsador era liberado de nuevo, el LED volvía a encender.

Conclusión

Los resultados obtenidos a partir del sistema fueron los buscados. El LED se prendía y se apagaba según las especificaciones pedidas, mostrando adecuadamente el funcionamiento del código y del circuito. Esto muestra nuestra introducción exitosa al lenguaje de programación C++ en conjunto de nuestros aprendizajes sobre la conexión de circuitos.

Por otro lado, el uso del pulsador probó ser un desafío, puesto que nuestro equipo no estaba muy familiarizado con este. Tuvimos que aprender desde cero la manera de conectar el componente y hacer ajustes en el boceto de la esquemática.

Por último, cabe mencionar las dificultades que desafiamos al momento de pasar la esquemática del sistema digital a físico. Como ya ha sido mencionado, el programa utilizado para el diseño del circuito cuenta con limitaciones que tuvieron que ser atendidas posteriormente. Esto, a pesar de no ser particularmente complicado, nos notificó de posibles problemas que pudiéramos enfrentar en el futuro durante la misma etapa en sistemas más complejos.

